

Esame OS Internals (Cabodi)

Data: 07/07/2023

Nota Generale: In tutte le domande, ogni volta che il risultato è una selezione, un SI/NO, o un numero, sono necessari una motivazione/spiegazione o passaggi intermedi che portino al risultato.

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Sostituzione Pagine e Memoria Virtuale)

Contesto: Si consideri la seguente stringa di riferimenti di memoria, per un determinato processo. Per ogni riferimento (Byte addressing, con indirizzi espressi in esadecimale) viene riportata anche l'operazione di lettura/scrittura (R/W):

R 33FB, R 1B64, W 30D3, W 237E, R 0AC8, W 23D7, R 174A, R 0965, W 32A0, R 1BB0, W 09E5, R 3380, R 2A94, R 11B8.

Si supponga che gli indirizzi fisici e logici siano su 16 bit, la dimensione della pagina sia 2KByte e 70FF sia l'indirizzo massimo utilizzabile dal programma (il limite superiore dello spazio degli indirizzi).

- A) Calcolare la dimensione dello spazio degli indirizzi (address space, espresso come numero di pagine) e la frammentazione interna.
- A) Calcolare la stringa di riferimenti di pagina.
- A) Simulare un algoritmo di sostituzione di pagine di tipo second-chance, con 3 frame disponibili. Rappresentare il resident set (frame fisici contenenti pagine logiche) dopo ogni riferimento in memoria. Indicare esplicitamente, per ogni frame allocato, il numero di pagina e il bit di riferimento (utilizzare la notazione p_r). Indicare anche i page fault. Si supponga che il bit di riferimento di una pagina sia inizializzato a 0 dopo il relativo page fault.

Domanda 2 (HDD, Geometria e Prestazioni)

Contesto: Si consideri un'unità disco rigido (HDD) con tecnologia e geometria standard. Il disco ha: dimensione del settore di 512 Byte, 2048 tracce per faccia, 50 settori per traccia, 5 dischi/piatti a doppia faccia, Tempo medio di seek di 10 msec.

- A) Qual è la capacità di una traccia (quanti Byte può contenere)? Qual è la capacità di ogni faccia? Qual è la capacità del disco? Quanti cilindri ha il disco?
- A) Il metodo di organizzazione di disco CHS (Cylinder-Head-Sector) consente a un blocco del disco di estendersi su più settori di una data traccia (identificata da una coppia cilindro, testina) ma non su tracce diverse. Le seguenti dimensioni dei blocchi sono valide? 256 B, 2048 B, 51200 B (rispondere per ognuna e motivare le risposte).

- A) Se i dischi ruotano a 5400 giri/min (giri al minuto), qual è la latenza rotazionale massima? Qual è la latenza rotazionale media? Si sa che è possibile trasferire un'intera traccia di dati per rotazione, qual è la velocità di trasferimento (espressa in bit/secondo)? Quale sarebbe la velocità di trasferimento se un intero cilindro di dati potesse essere trasferito in una rotazione?
-

Domanda 3 (File System, Allocazione e Archiviazione)

- A) Un'applicazione deve organizzare un file con un'intestazione (header) e più sezioni. L'header include, tra gli altri dati, offset e dimensioni di tutte le sezioni del file.
1. L'applicazione può supporre che l'header, così come ogni sezione inizi all'inizio di un blocco del disco (nel file system)? (Motivare)
 2. Può l'applicazione decidere di organizzare i dati in più file (anziché in un singolo file): un file per l'intestazione e uno per ogni sezione? (Se NO, motivare, se SI, motivare e spiegare come gli offset di sezione nell'intestazione devono essere rappresentati, codificati o modificati).
- A) Un file di testo viene copiato in tre diversi file system, con formati di allocazione file, rispettivamente: allocazione contigua (per multiplo di un blocco di disco), allocazione collegata (senza FAT), e allocazione basata su Inode. La dimensione di un blocco su disco è la stessa in tutti e tre i file system. Possiamo supporre che la frammentazione interna sarà la stessa in tutti i file system? (Rispondere e motivare).
- A) Un file system contiene tre file di testo `a.txt`, `b.txt` e `c.txt`. Un'applicazione di archiviazione/compressione di file (come ad esempio gzip, tar, 7z, rar, ecc.) memorizza il (contenuto dei) tre file di testo all'interno di un singolo file di archivio (ad esempio `abc.zip`).
1. I tre file di testo possono condividere lo stesso blocco del disco, il che significa che il file system li memorizza (almeno parzialmente) all'interno dello stesso blocco del disco? (Motivare)
 2. È possibile, per il contenuto dei tre file di testo, replicati e/o compressi nel file di archivio, condividere (almeno parzialmente) lo stesso blocco disco (in modo da essere memorizzato all'interno dello stesso blocco disco)? (Motivare)

2 SECTION 2: OS161

Domanda 4 (File System Framework in OS161)

Contesto: E' dato un sistema OS161. Si consideri il framework del file system utilizzato in LAB5, parzialmente rappresentato dai seguenti frammenti di codice (da `proc.h` e `file_syscalls.c`):

```
struct proc {
#ifdef OPT_FILE
    /* per-process open file table */
    struct openfile *fileTable[OPEN_MAX];
#endif
};

/* system open file table */
struct openfile {
    struct vnode *vn;
    off_t offset;
    unsigned int countRef;
};

struct openfile systemFileTable[SYSTEM_OPEN_MAX];
```

Rispondere alle seguenti domande:

- B) Perché `systemFileTable` è un array di `struct openfile`, mentre `fileTable` è un array di puntatori a `struct openfile`?
 - B) Il campo `countRef` è ridondante, considerando che il `vnode` puntato dal campo `vn` ha (internamente) il suo contatore di riferimenti?
 - B) Nel caso in cui sia necessario supportare operazioni di lock dei file, cosa andrebbe scelto tra uno spinlock e un lock e dove dovrebbe essere posizionato (o posizionati, se multipli): in `systemFileTable`, in `fileTable` o altrove?
 - B) Qual è il possibile ruolo delle funzioni `copyin` e `copyout` nell'implementazione delle chiamate di sistema `sys_read` e `sys_write`?
-

Domanda 5 (Sincronizzazione e Spinlock)

Contesto: Si considerino, in OS161, le due possibili implementazioni (parziali) di `spinlock_acquire` mostrate di seguito:

```
void spinlock_acquire(struct spinlock *splk) {
    // ...
    while (1) {
        if (spinlock_data_get(&splk->splk_lock) != 0) {
            continue;
        }
        if (spinlock_data_testandset(&splk->splk_lock) != 0) {
            continue;
        }
    }
}
```

```

        break;
    }
    // ...
}

void spinlock_acquire2(struct spinlock *splk) {
    // ...
    while (1) {
        while (spinlock_data_get(&splk->splk_lock) != 0);
        if (spinlock_data_testandset(&splk->splk_lock) == 0) {
            continue;
        }
    }
    // ...
}

```

Rispondere alle seguenti domande:

- B) Perché `spinlock_acquire` utilizza sia `spinlock_data_get` che `spinlock_data_testandset`, invece di chiamarne solo una?
- B) `spinlock_acquire2` equivale a `spinlock_acquire`? Se sì, motivarlo, se no, motivarlo e, se possibile, modificarla per renderla equivalente.

SOLUZIONI

Domanda 1 (Sostituzione Pagine e Memoria Virtuale)

A) Spazio degli indirizzi e Frammentazione Interna:

L'architettura prevede indirizzi fisici e logici su 16 bit, con paginazione al byte. Il limite superiore dello spazio logico utilizzabile è `0x70FF`. Convertendo questo valore esadecimale in decimale:

$$0x70FF = 7 \times 16^3 + 0 \times 16^2 + 15 \times 16 + 15 = 28672 + 255 = 28927$$

Poiché l'indirizzamento parte da 0, lo spazio totale utilizzato dal programma è di 28928 Byte. Essendo la pagina di 2 KB (2048 Byte), il numero di pagine logiche necessarie si ottiene dividendo lo spazio per la dimensione della pagina e arrotondando per eccesso:

$$\text{Numero Pagine} = \lceil \frac{28928}{2048} \rceil = \lceil 14,125 \rceil = \mathbf{15 \text{ pagine}}$$

Le 15 pagine sono numerate da 0 a 14. L'ultima pagina (Pagina 14) inizia esattamente all'indirizzo $14 \times 2048 = 28672$. I byte effettivamente utilizzati all'interno di quest'ultima pagina vanno da 28672 all'indirizzo massimo 28927, per un totale di $28927 - 28672 + 1 = 256$ Byte.

La **frammentazione interna** è data dallo spazio sprecato (allocato ma non utilizzato) all'interno dell'ultima pagina:

$$\text{Frammentazione Interna} = 2048 - 256 = \mathbf{1792 \text{ Byte}}$$

B) Stringa di riferimenti di pagina:

La dimensione della pagina di 2 KB (2^{11} Byte) implica che i primi 5 bit dell'indirizzo a 16 bit indichino il numero di pagina, e i restanti 11 bit rappresentino l'offset. In esadecimale, la traduzione corretta dei 14 indirizzi produce la seguente stringa di riferimenti: **6, 3, 6, 4, 1, 4, 2, 1, 6, 3, 1, 6, 5, 2**.

C) Simulazione Algoritmo Second-Chance (Clock):

Stato iniziale: 3 frame vuoti.

Notazione utilizzata: [Pagina(Bit)].

Il simbolo (*) indica la posizione del puntatore (hand) prima di processare il riferimento in ingresso.

Rif	Stato Frame	Azione ed Eventuale Page Fault
R 6	(*)[- , - , -]	Fault (1). Inserisco 6. Ptr avanza.
R 3	[6(0),(*) - , -]	Fault (2). Inserisco 3. Ptr avanza.
W 6	[6(0), 3(0),(*) -]	Hit! Aggiorno bit di 6. Ptr fermo.
W 4	[6(1), 3(0),(*) -]	Fault (3). Inserisco 4. Array pieno. Ptr a 0.
R 1	(*)[6(1), 3(0), 4(0)]	Fault (4). Ptr su 6(1)->diventa 0, avanzo a 1.
		Ptr su 3(0)->Sfratto 3! Inserisco 1. Ptr a 2.
W 4	[6(0), 1(0),(*)4(0)]	Hit! Aggiorno bit di 4. Ptr fermo.
R 2	[6(0), 1(0),(*)4(1)]	Fault (5). Ptr su 4(1)->diventa 0, avanzo a 0.
		Ptr su 6(0)->Sfratto 6! Inserisco 2. Ptr a 1.
R 1	[2(0),(*)1(0), 4(0)]	Hit! Aggiorno bit di 1. Ptr fermo.
W 6	[2(0),(*)1(1), 4(0)]	Fault (6). Ptr su 1(1)->diventa 0, avanzo a 2.
		Ptr su 4(0)->Sfratto 4! Inserisco 6. Ptr a 0.
R 3	(*)[2(0), 1(0), 6(0)]	Fault (7). Ptr su 2(0)->Sfratto 2! Inserisco 3.
W 1	[3(0),(*)1(0), 6(0)]	Hit! Aggiorno bit di 1. Ptr fermo.
R 6	[3(0), 1(1),(*)6(0)]	Hit! Aggiorno bit di 6. Ptr fermo.
R 5	[3(0),(*)1(1), 6(1)]	Fault (8). Ptr su 1(1)->0. Ptr su 6(1)->0.
		Ptr su 3(0)->Sfratto 3! Inserisco 5. Ptr a 1.
R 2	[5(0),(*)1(0), 6(0)]	Fault (9). Ptr su 1(0)->Sfratto 1! Inserisco 2.

Risultato Finale: 9 Page Fault Totali.

Domanda 2 (HDD, Geometria e Prestazioni)

Dati Iniziali:

Settore = 512 B; Tracce/faccia = 2048; Settori/traccia = 50; Facce totali = 10 (5 piatti a doppia faccia); RPM = 5400.

A) Calcolo Geometria del Disco:

- **A.1) Capacità Traccia:** Settori per traccia $\times$ Capacità settore = $50 \times 512 \text{ B} = 25.600 \text{ B}$ (25 KB).
- **A.2) Capacità Faccia:** Tracce per faccia $\times$ Capacità traccia = $2048 \times 25.600 \text{ B} = 52.428.800 \text{ B}$ ($\sim 50 \text{ MB}$).
- **A.3) Capacità Disco Totale:** Numero di facce $\times$ Capacità faccia = $10 \times 52.428.800 \text{ B} = 524.288.000 \text{ B}$ ($\sim 500 \text{ MB}$).
- **A.4) Numero di Cilindri:** Un cilindro è l'insieme logico delle tracce poste allo stesso raggio su tutte le facce. Il numero di cilindri coincide con il numero di tracce per singola faccia: **2048 cilindri**.

B) Validità Dimensioni Blocchi (Vincolo CHS):

Affinché un blocco logico sia valido in organizzazione CHS, deve rispettare due regole fondamentali: 1. Essere un *multiplo intero della dimensione del settore* (512 B), poiché l'hardware non indirizza unità inferiori. 2. Essere contenuto interamente in una singola traccia (dimensione totale $\leq 25.600 \text{ B}$).

- **256 B: NON VALIDO.** È strettamente minore di un settore fisico (512 B), violando l'unità minima di trasferimento.
- **2048 B: VALIDO.** È un multiplo del settore ($2048/512 = 4$ settori) e la sua estensione (4 settori) è ampiamente contenuta nella capacità della singola traccia (50 settori).
- **51.200 B: NON VALIDO.** Pur essendo un multiplo del settore ($51.200/512 = 100$ settori), la sua estensione eccede la capacità fisica di una singola traccia (100 settori richiesti > 50 settori disponibili), obbligando uno *spill* sulla traccia/cilindro successivo, violando il vincolo CHS.

C) Calcolo Latenze e Transfer Rate (a 5400 RPM):

Velocità angolare in Giri al Secondo (RPS) = $5400/60 = 90$ giri/sec.

- **Latenza Rotazionale Max:** È il tempo per compiere una rotazione completa.
 $t_{max} = \frac{1}{90} \text{ s} \approx \mathbf{11,11 \text{ ms.}}$
- **Latenza Rotazionale Media:** È statisticamente la metà della latenza massima.
 $t_{media} = \frac{1}{2 \times 90} = \frac{1}{180} \text{ s} \approx \mathbf{5,55 \text{ ms.}}$
- **Transfer Rate (Traccia):** Trasferendo un'intera traccia (25.600 B) ad ogni rotazione:
 $TR_{traccia} = 25.600 \text{ B/giro} \times 90 \text{ giri/sec} \times 8 \text{ bit/B} = \mathbf{18.432.000 \text{ bit/s}}$ ($\sim 18,4 \text{ Mbps}$).
- **Transfer Rate (Cilindro):** Leggendo in parallelo da tutte le 10 testine contemporaneamente, la capacità trasferita in una singola rotazione è quella di 10 tracce ($25.600 \text{ B} \times 10 = 256.000 \text{ B/giro}$):
 $TR_{cilindro} = 256.000 \text{ B/giro} \times 90 \text{ giri/sec} \times 8 \text{ bit/B} = \mathbf{184.320.000 \text{ bit/s}}$ ($\sim 184,3 \text{ Mbps}$).

Domanda 3 (File System, Allocazione e Archiviazione)

A) Organizzazione file, Header e Sezioni:

- **1. Si può supporre che l'header o le sezioni inizino all'inizio di un blocco? NO.**
Motivazione: I sistemi operativi forniscono alle applicazioni l'astrazione del file come un **flusso contiguo di byte (byte stream)**, nascondendo la reale divisione in blocchi fisici. Se l'applicazione scrive un header di 100 byte, la sezione successiva inizierà al byte logico 101, che il SO mapperà nel medesimo blocco fisico dell'header. Per far allineare le strutture ai blocchi fisici (es. da 4KB), l'applicazione deve forzare l'allineamento inserendo esplicitamente dei byte di riempimento (*padding*) fino al raggiungimento del limite del blocco.
- **2. È possibile usare più file per header e sezioni? SI.**
Motivazione: Invece di usare offset fisici all'interno di un unico flusso, la logica di indirizzamento diventa di livello superiore. Gli offset nell'header non possono più essere semplici interi, ma devono essere convertiti in tuple o identificatori strutturati, tipicamente nella forma (Nome_File, Offset_Interno). Se ogni file aggiuntivo contiene esattamente una sezione, il valore di Offset_Interno sarà implicitamente 0.

B) Frammentazione interna in Allocazione Contigua, Linked List e Inode:

- **Possiamo supporre che sia la stessa? NO.**
Motivazione: La frammentazione interna dipende da quanti byte di payload effettivo possono essere ospitati in ogni blocco. Nell'allocazione Contigua e a Inode, l'intero blocco logico (es. 4096 byte) è destinato ai dati utente. Nell'allocazione Collegata (Linked List) senza FAT, una porzione del blocco (es. 4 byte) deve essere riservata per memorizzare il

puntatore al blocco successivo. Avendo una capacità utile (payload) per blocco differente, il medesimo file richiederà un numero diverso di blocchi per essere memorizzato nei diversi FS. Di conseguenza, il residuo di byte inutilizzati nell'ultimo blocco (*frammentazione interna*) sarà matematicamente diverso.

C) Archiviazione e Condivisione dei Blocchi:

- **1. I file originali a.txt, b.txt e c.txt possono condividere lo stesso blocco? NO.**

Motivazione: In un file system tradizionale, il blocco fisico è l'unità atomica di allocazione, assegnata in modo esclusivo a un singolo i-node/file. Se un file non riempie completamente il suo blocco, lo spazio rimanente è perso in frammentazione interna e non può essere utilizzato per memorizzare dati appartenenti ad altri file distinti.

- **2. I contenuti all'interno dell'archivio abc.zip possono condividere lo stesso blocco? SI.**

Motivazione: Dal punto di vista del File System, l'archivio abc.zip è un **singolo file** continuo. L'applicazione user-space (es. tar/zip) serializza e accoda i byte compressi di a, b e c in un unico byte stream ininterrotto. Di conseguenza, i dati di file logicamente diversi verranno scritti dal SO in modo adiacente all'interno dei medesimi blocchi fisici allocati per abc.zip.

Domanda 5 (Sincronizzazione e Spinlock)

A) Perché spinlock_acquire utilizza sia get che testandset?

L'implementazione proposta utilizza il pattern noto come **Test-and-Test-and-Set (TTAS)**. L'istruzione hardware `testandset` (essendo un'operazione atomica di Read-Modify-Write) ha un costo enorme in sistemi multiprocessore: ogni invocazione invalida la linea di cache corrispondente sulle altre CPU e genera traffico sul bus di memoria (bus locking o cache coherence traffic). Se i thread in attesa invocassero in loop continuo `testandset`, saturerebbero il bus degradando le prestazioni dell'intero sistema. Invocare preventivamente `spinlock_data_get()` permette al thread di "girare a vuoto" (spin) leggendo semplicemente dalla propria **cache locale** finché il lock risulta occupato, azzerando il traffico sul bus. Solo quando la `get` legge uno 0, il thread invoca la `testandset` per tentare l'acquisizione reale.

B) Equivalenza e correzione di spinlock_acquire2:

Le due funzioni NON sono equivalenti. La funzione `spinlock_acquire2` contiene un grave errore logico (oltre all'assenza del `break` di uscita).

- L'istruzione `testandset` restituisce il valore *precedente* della variabile.
- Se ritorna 0, l'acquisizione ha avuto **successo** (il lock era libero ed è stato settato).
- Se ritorna `!= 0`, l'acquisizione ha **fallito** (il lock era già occupato).

Nell'implementazione `spinlock_acquire2`, l'istruzione `if (spinlock_data_testandset(...) == 0) { continue; }` forza il thread a scartare l'acquisizione appena avvenuta con successo e a ricominciare il ciclo.

Per renderla equivalente alla prima versione, bisogna invertire la logica di controllo, interrompendo il ciclo (`break`) esclusivamente quando il ritorno della `testandset` è uguale a 0:

```
void spinlock_acquire2_correct(struct spinlock *splk) {
    // ...
    while (1) {
```

```

// Spin a basso costo sulla cache locale
while (spinlock_data_get(&splk->splk_lock) != 0);

// Tentativo di acquisizione atomica
if (spinlock_data_testandset(&splk->splk_lock) == 0) {
    break; // SUCCESS: il lock era 0, ora è 1. Usciamo dal ciclo.
}
// FAIL: qualcun altro ci ha preceduto.
// Il while(1) fa ricominciare il ciclo.
}
// ...
}

```